Craft Atelier Fulfillment Engine

Overview

The **Craft Atelier Fulfillment Engine** is an enterprise-grade order management and delivery orchestration platform tailored for artisan workspaces and premium craft businesses. Designed to streamline end-to-end fulfillment, it seamlessly connects back-office operations with on-the-floor execution.

Key Enterprise Features

- **Real-time Tablet Operations (WebSocket auto-updating)**: Empower your floor staff and delivery teams with live, auto-updating dashboards powered by WebSocket technology. Order status changes are reflected instantly across all devices without manual refreshes.
- **Bulletproof Financial Snapshots**: Keep track of operations with immutable, precision-engineered financial snapshots, ensuring that transaction states and invoice amounts are perfectly captured at the exact moment of fulfillment.
- **Chain of Custody Audit Logging**: Every item's journey from assembly to delivery is meticulously recorded, providing full transparency, accountability, and strict traceability at every touchpoint.
- **Secure Delivery PIN (OTP) Verification**: Guarantee that premium goods end up in the right hands. Customers receive a One-Time Password (OTP) required for delivery handover, virtually eliminating misdeliveries and fraud.
- **SMS Webhook Auto-Reconciliation**: Integrate seamlessly with external SMS gateways. Inbound delivery confirmations and payment updates automatically reconcile with back-end orders via robust webhooks and automated M-Pesa routines.

Architecture

The platform features responsive, tablet-optimized HTML views tailored for high-speed fulfillment environments, integrating securely with backend Supabase services via WebSockets for real-time state and REST APIs for secure OTP/SMS webhook routing.

📦 Standard Operating Procedure: Warehouse & Admin Hub

🎯 Overview

This Standard Operating Procedure (SOP) outlines the updated workflows for the Fulfillment Station and Admin Headquarters. All systems are now synchronized via real-time databases, minimizing manual refreshes and ensuring strict chain-of-custody through Delivery PIN verification.

1️⃣ Fulfillment Station (Ops Packing)

Real-Time Order Queue

- **No Manual Refreshes Needed:** The packing station tablets are now equipped with real-time syncing. Whenever an order is released by finance, it will automatically appear in your queue. The "CONNECTING..." badge will change to show active queue counts dynamically.

BOM Packing Verification

1. Review the Customer Card and Order requirements on your screen.
2. Carefully pack the items, checking off each required component on the screen (e.g., Curated Yarn Bundles, Ergonomic Hook, Notions Pack, etc.).
3. **Verify All Items & Seal Box:** Once all checkboxes (including the Bonus Charm) are ticked, click this button to stage the order for dispatch.

Exception Handling: Report Stock Issue

- If you realize a required component is missing or damaged, **DO NOT** verify the order.
- Click the red **"Report Stock Issue"** button.
- This will immediately set the order status to `ON_HOLD` and remove it from your active packing queue, alerting management to rectify the inventory problem.

2️⃣ Dispatch & Rider Hand-off

Assigning a Carrier

4. Once an order is `STAGED_FOR_DISPATCH`, it will move to the Handover view.
5. Click **"Assign Carrier / Hand Off"**.
6. Select the active motorbike rider from the dropdown menu and confirm dispatch.

WhatsApp Notification & Delivery PIN

- Upon hand-off, the system generates a secure **4-digit Delivery PIN**.
- A WhatsApp Click-to-Chat window will automatically open on your device.
- **Action Required:** Send the pre-filled message to the customer. It contains the rider's contact details and their unique Delivery PIN, which they must provide upon arrival.

3️⃣ Tracking & Delivery Confirmation (Admin Hub)

Order Tracking Board

- The Admin Hub features a live tracking board filterable by status (`QUEUED_FOR_PACKING`, `PACKING_IN_PROGRESS`, `STAGED_FOR_DISPATCH`, `IN_TRANSIT`, `DELIVERED`).
- Admins can monitor real-time fulfillment progression across the warehouse.

Completing a Delivery (PIN Required)

- Once the rider reaches the customer, they must ask the customer for their 4-digit Delivery PIN.
- On the Admin Hub (or via Rider App), click **"Complete Delivery"** on an `IN_TRANSIT` order.
- **Mandatory Step:** You **MUST** enter the exact 4-digit Delivery PIN.
- *If the PIN is correct:* The order is marked `DELIVERED` and logged in the audit trail.
- *If the PIN is incorrect:* The system will reject the status change. You must confirm the correct PIN with the customer.

4️⃣ Catalog & Inventory Management

Adding New Products

Admins can manage the product catalog directly from the Admin Hub's Catalog & Inventory section. When clicking **"+ Add New Kit"**, you will be prompted to choose between two methods:

7. **Single Kit (Option '1'):** Add a kit manually by typing in the Title, Price (in KES), and Initial Inventory Count one by one.
8. **Bulk Import via JSON (Option '2'):** Instantly add multiple products by pasting a JSON array of kits.

- *Example format:* `[{"title": "Beginner Kit", "price_cents": 250000, "inventory_count": 15}, {"title": "Advanced Kit", "price_cents": 450000, "inventory_count": 5}]`
- This is highly recommended for seasonal catalog updates or initial data population.

Editing Existing Products

- Click **"Edit"** next to any kit to update its inventory count or price. The system will safely archive the old record and create a new version to maintain strict financial and inventory history.

Craft Atelier Fulfillment - Technical Architecture

Overview

The platform is a web-based e-commerce and order fulfillment system for "Craft Atelier" DIY kits. It connects a storefront with an admin control hub and a live warehouse packing station. The backend leverages Supabase (PostgreSQL) for real-time state synchronization, authentication, and database management.

Technical Stack

- **Frontend**: HTML5, CSS (vanilla/custom CSS), JavaScript (vanilla)
- **Backend / Database**: Supabase (PostgreSQL, Supabase JS Client v2)
- **Real-time Engine**: Supabase Realtime (WebSocket `postgres_changes` listener)
- **Automation Triggers**: Android SMS Forwarder

Architecture Components

1. Storefront (`storefront.html`)

The customer-facing application where users browse the catalog and place orders.

- **Dynamic M-Pesa Till**: Fetches the active Paybill/Till number dynamically from the `business_settings` table.
- **Order Placement**: Captures customer details, calculates subtotal, delivery fee, and total costs before inserting a `PENDING_PAYMENT` record into the `orders` table.

2. Admin Hub (`admin-hub.html`)

The central command center for operators to verify payments, manage inventory, and oversee the fleet.

- **Financial Verification**: Allows admins to cross-check M-Pesa references and manually release orders to the warehouse queue (`QUEUED_FOR_PACKING`).
- **Fleet & Staff Management**: Toggle rider and staff access states.
- **Order Tracking Board**: Visualizes the entire pipeline from `PENDING_PAYMENT` to `DELIVERED`. Supports completing deliveries via a PIN check.
- **Catalog Management**: Provides UI to add/edit kits (supports JSON bulk import) and manage inventory.

3. Ops Packing Station (`ops-packing.html`)

A live tablet/mobile-friendly UI for warehouse staff (packers and dispatchers).

- **Real-time Queue**: Subscribes to Supabase `postgres_changes` on the `orders` table. Automatically updates the UI when an order becomes `QUEUED_FOR_PACKING` or `STAGED_FOR_DISPATCH`.

- **BOM Verification**: Packers must check off required Bill of Materials (BOM) items before sealing the box. Triggers the `verify_and_stage_order` RPC.
- **Dispatch**: Dispatchers assign an active rider, triggering the `dispatch_order` RPC, which generates a WhatsApp notification link containing the Delivery PIN.

Critical New Additions

1. Order Audit Logging

Every critical state change is tracked for accountability in the `order_audit_logs` table.

- **Fields**: `order_id`, `old_status`, `new_status`, `changed_by`, `notes`.
- **Example**: Moving an order from `IN_TRANSIT` to `DELIVERED` logs the change, noting whether it was done by Admin or the Rider App, ensuring full operational traceability.

2. Order Financial Snapshots

The `orders` table now strictly captures the financial state at the time of checkout to prevent historical data mutation if catalog prices change:

- `subtotal_cents`: Sum of kit prices.
- `delivery_fee_cents`: Cost of the selected delivery method.
- `total_cents`: Final expected payment amount.

3. Delivery PIN System

To prevent theft and misdelivery:

9. **Generation**: When an order is handed to a rider (dispatched), a random 4-digit `delivery_pin` is generated and saved to the `orders` table.
10. **Distribution**: The PIN is sent to the customer via a pre-filled WhatsApp message.
11. **Verification**: The rider must collect this PIN from the customer. The Admin/Rider App checks this PIN against the database before marking the order as `DELIVERED`.

4. Real-Time WebSocket Synchronization

The Ops Packing UI utilizes Supabase Realtime channels to eliminate manual refreshing. When an order state changes, the UI automatically fetches the next order if it matches the current user's role (packer/dispatcher):

```
client.channel('ops-orders-channel')
  .on('postgres_changes', { event: 'UPDATE', schema: 'public',
table: 'orders' }, payload => {
      // Auto-fetch next order when state changes match the
user's role
  }).subscribe();
```

5. M-Pesa Automation via PL/pgSQL

An automated M-Pesa verification flow is supported via an Android SMS Forwarder app.

- **Mechanism**: The Android app listens for M-Pesa confirmation SMS messages and forwards them to the backend database.
- **Function**: The `auto_verify_mpesa_payment` (PL/pgSQL) function is designed to be hit by this forwarder. It parses the reference code, checks the expected amount, matches it against a `PENDING_PAYMENT` order, and automatically advances its status to `QUEUED_FOR_PACKING` upon success.

6. Business Settings Dictionary

The `business_settings` table acts as a dynamic key-value store for global configurations, avoiding hardcoded constants in the front-end.

- **Key Example**: `mpesa_till`
- **Usage**: The Storefront and Admin Hub pull this dynamically, ensuring the business can update operational details like the Till number in real-time without redeploying the site code.